

Microcontroladores: Laboratorio 2

1st Hector Pereira

Ingeniería en Mecatrónica

Universidad Tecnológica (UTEC)

Fray Bentos, Uruguay

hector.pereira@estudiantes.utec.edu.uy

2nd Isaac Martirena

Ingeniería en Mecatrónica

Universidad Tecnológica (UTEC)

Fray Bentos, Uruguay

isaac.martirena@estudiantes.utec.edu.uy

Resumen—

Keywords:

I. INTRODUCCIÓN

I-A. Propósito del laboratorio

I-B. Descripción general de los módulos

I-C. Competencias desarrolladas

I-D. Cierre o enfoque global

II. MARCO TEÓRICO

II-A. Procesamiento de señales y sensores de color

Un sensor de luz o *Light Dependent Resistor* (LDR) es un dispositivo fotoresistivo cuya resistencia eléctrica varía de manera inversamente proporcional a la intensidad luminosa incidente. En condiciones de baja iluminación, la resistencia del material semiconductor (generalmente sulfuro de cadmio) es elevada, mientras que ante una mayor cantidad de luz, la energía de los fotones libera más portadores de carga, reduciendo la resistencia. Este principio permite convertir la intensidad luminosa en una señal eléctrica analógica, que puede ser medida por el convertidor analógico-digital (ADC) del microcontrolador.

Para representar y clasificar los colores percibidos por el sensor se emplea el modelo aditivo de color **RGB** (Red, Green, Blue), basado en la combinación de tres componentes de luz primaria. Cada componente puede tomar valores dentro de un rango de 0 a 255, generando un espacio tridimensional en el cual cada punto (R, G, B) define un color único. En este espacio, los ejes representan la contribución relativa de cada componente: el rojo sobre el eje R , el verde sobre el eje G y el azul sobre el eje B . De esta forma, el color blanco corresponde a $(255, 255, 255)$ y el negro a $(0, 0, 0)$.

Dado que la respuesta espectral del LDR no es lineal ni uniforme para todos los colores, es necesario un proceso de **calibración**. Este consiste en medir los valores del ADC frente a una hoja de referencia con colores conocidos, obteniendo así un conjunto de vectores (R_i, G_i, B_i) asociados a cada color de muestra. Durante la operación, el sistema

compara el vector medido (R_m, G_m, B_m) con los valores almacenados y determina el color más cercano calculando la *distancia euclidiana* entre ellos:

$$d_i = \sqrt{(R_m - R_i)^2 + (G_m - G_i)^2 + (B_m - B_i)^2}$$

El color correspondiente al menor valor de d_i se considera el identificado. Este método permite una clasificación robusta frente a pequeñas variaciones de iluminación o de lectura analógica, asegurando una detección estable y repetible en entornos experimentales.

II-B. Señales PWM y control de servomotores

El *Pulse Width Modulation* (PWM) o modulación por ancho de pulso es una técnica ampliamente utilizada en sistemas embebidos para controlar dispositivos analógicos mediante señales digitales. Consiste en generar una señal periódica cuya frecuencia permanece constante, pero cuyo ciclo de trabajo (*duty cycle*) varía según la magnitud que se desea representar. Este ciclo de trabajo, definido como la relación entre el tiempo en nivel alto (t_{on}) y el periodo total (T), se expresa como:

$$D = \frac{t_{on}}{T} \times 100 (\%)$$

En el caso de los servomotores de tipo hobby, el control de posición se logra variando el ancho del pulso dentro de un periodo fijo de aproximadamente 20 ms (equivalente a una frecuencia de 50 Hz). Un pulso de 1 ms suele corresponder a una posición angular mínima (por ejemplo, 0°), mientras que un pulso de 2 ms representa la posición máxima (180°). De esta manera, el ángulo del eje del servo se determina de forma proporcional al ancho del pulso aplicado a su señal de control.

En el microcontrolador ATmega328P, la generación de esta señal se realiza mediante el **Timer/Counter1**, un temporizador de 16 bits capaz de operar en modo de comparación rápida (*Fast PWM*). Configurando los registros TCCR1A y TCCR1B para emplear el modo PWM con un prescaler de 8 y un valor superior (ICR1) de 39999, se obtiene un periodo de 20 ms bajo una frecuencia de reloj de 16 MHz. El registro

de comparación OCR1A define el tiempo en alto de la señal, ajustando el ángulo del servomotor según la ecuación:

$$t_{on} = 1000 \mu s + \left(\frac{\text{ángulo}}{180} \right) \times 1000 \mu s$$

De esta forma, el módulo PWM permite un control preciso y estable del servomotor, sin necesidad de intervención constante del procesador, ya que el hardware del temporizador mantiene la generación continua de los pulsos.

II-C. Reproducción de sonido digital

El sonido es una onda mecánica periódica cuya frecuencia determina la percepción del tono musical. En términos físicos, la frecuencia (f), medida en hertzios (Hz), representa el número de oscilaciones por segundo, mientras que su inverso define el periodo ($T = 1/f$). En el contexto de un buzzer piezoeléctrico, este principio se aprovecha generando una señal eléctrica alternante que provoca vibraciones en el material piezoeléctrico, transformándolas en ondas sonoras audibles.

En un sistema basado en microcontroladores, dichas oscilaciones pueden producirse mediante los temporizadores internos (*timers*), configurados para generar ondas cuadradas con una frecuencia ajustable. Por ejemplo, al programar un temporizador en modo de comparación (CTC o *Clear Timer on Compare Match*), el registro de comparación (OCR_x) se calcula en función de la frecuencia deseada y del reloj del sistema (f_{clk}) según:

$$OCR_x = \frac{f_{clk}}{2 \times N \times f_{sonido}} - 1$$

donde N es el factor de prescalado. Cada vez que el temporizador alcanza este valor, se invierte el estado del pin de salida, generando así una onda cuadrada que excita directamente al buzzer. De esta forma, variando el valor de OCR_x, es posible reproducir distintas notas musicales.

Para crear melodías, se utiliza una estructura de datos que almacena pares de valores de frecuencia y duración asociados a cada nota. El microcontrolador recorre secuencialmente este arreglo, configurando el temporizador para cada frecuencia y manteniéndola activa durante el tiempo correspondiente. Este procedimiento puede implementarse de manera eficiente mediante interrupciones del temporizador, lo que permite reproducir secuencias musicales sin bloquear la ejecución del resto de las tareas del sistema.

El resultado es una **melodía digital**, donde cada nota se sintetiza a partir de una señal cuadrada controlada por software, imitando el comportamiento de un oscilador simple y demostrando la capacidad del microcontrolador para generar sonidos a partir de la modulación temporal de sus recursos internos.

II-D. Interfaz de usuario e interacción hombre-máquina

En los sistemas embebidos, la **interfaz de usuario** (*User Interface*, UI) constituye el conjunto de mecanismos físicos y lógicos que permiten la comunicación entre el usuario y el dispositivo. Su función principal es traducir las acciones del operador (entradas) en respuestas visibles o audibles (salidas) que reflejen el estado actual del sistema. Este intercambio bidireccional resulta fundamental para lograr un funcionamiento intuitivo, seguro y eficiente en aplicaciones donde la interacción humana es constante.

En el caso del sistema desarrollado, la interfaz está compuesta por un teclado matricial, una pantalla LCD, indicadores luminosos y un buzzer. El teclado permite el ingreso de datos —como contraseñas o comandos—, mientras que la pantalla LCD presenta mensajes de guía o confirmación. Los LEDs y el buzzer complementan la experiencia de usuario al proporcionar *feedback* visual y auditivo, respectivamente, facilitando la interpretación inmediata de los estados del sistema (éxito, error o alarma).

La lógica de interacción se organiza mediante una **máquina de estados finitos** (FSM, por sus siglas en inglés), estructura ampliamente utilizada en el diseño de interfaces embebidas. En este enfoque, el programa se divide en estados bien definidos, tales como *inicio*, *ingreso*, *verificación* o *alarma*, y las transiciones entre ellos se producen en función de eventos específicos, como la presión de una tecla o la comparación de contraseñas. Este modelo favorece un código más claro, modular y predecible, al mismo tiempo que simplifica la gestión de entradas concurrentes y condiciones de error.

La combinación de estímulos visuales (mensajes y LEDs) y auditivos (tonos del buzzer) proporciona al usuario una retroalimentación inmediata sobre el resultado de sus acciones. Este tipo de *feedback* sensorial no solo mejora la usabilidad del sistema, sino que también incrementa su confiabilidad operativa, al minimizar ambigüedades y facilitar la detección de fallos o estados anómalos durante la interacción.

II-E. Almacenamiento no volátil y EEPROM

La **EEPROM** (*Electrically Erasable Programmable Read-Only Memory*) es un tipo de memoria no volátil integrada en la mayoría de los microcontroladores AVR, cuya principal característica es conservar los datos almacenados incluso cuando el sistema se apaga o se reinicia. A diferencia de la memoria RAM, que se borra al interrumpirse la alimentación, la EEPROM permite mantener información crítica de configuración o de usuario entre sesiones de uso, lo que la convierte en un recurso esencial para aplicaciones que requieren persistencia de datos.

En el caso del microcontrolador ATmega328P, la EEPROM posee una capacidad de 1 kB, organizada en celdas de 8 bits. Cada celda puede ser escrita y borrada eléctricamente un número limitado de veces, típicamente en el orden de 10^5 ciclos, según lo especificado en el *datasheet*

del dispositivo. Por este motivo, es importante administrar cuidadosamente las operaciones de escritura para evitar el desgaste prematuro de las celdas de memoria. El acceso a la EEPROM se realiza mediante instrucciones específicas del microcontrolador o a través de las funciones provistas por la librería `<avr/eeprom.h>` de AVR-LibC, que abstraen los procesos de lectura y escritura en bloques o bytes individuales.

Dentro del sistema desarrollado, la EEPROM se utiliza para el **almacenamiento persistente de contraseñas**. Al iniciar el dispositivo, el programa lee desde esta memoria la última contraseña registrada; si los datos no existen o no son válidos, se inicializa con una clave predeterminada. Cuando el usuario cambia su contraseña a través de la interfaz, el nuevo valor se escribe en la EEPROM, garantizando que permanezca disponible después de un apagado. Este mecanismo asegura tanto la continuidad funcional del sistema como la independencia de la memoria volátil, permitiendo que la información sensible se mantenga protegida y accesible de manera confiable a lo largo del tiempo.

II-F. Planificación temporal y multitarea cooperativa

En los sistemas embebidos, la ejecución simultánea de múltiples procesos suele requerir una estrategia de **planificación temporal** que permita atender diversas tareas sin provocar bloqueos ni retardos innecesarios. Una de las técnicas más utilizadas en microcontroladores de propósito general, como el ATmega328P, es la **multitarea cooperativa**, en la cual cada tarea se ejecuta de forma secuencial dentro del ciclo principal (`main loop`) y cede voluntariamente el control al resto una vez completada su función. A diferencia de la multitarea con planificación por interrupciones o sistemas operativos en tiempo real (RTOS), este enfoque no requiere un cambio de contexto ni la gestión de múltiples pilas, lo que lo hace ideal para aplicaciones de baja complejidad y recursos limitados.

El núcleo de esta técnica se basa en un **contador global de tiempo**, comúnmente implementado mediante un temporizador de hardware (*timer*) que se incrementa de forma periódica por interrupción. Este contador —por ejemplo, la variable `millis`— representa el tiempo transcurrido desde el arranque del sistema y permite medir intervalos de manera no bloqueante. Cada tarea compara el tiempo actual con el instante en que fue ejecutada por última vez; si la diferencia supera el periodo asignado, la función correspondiente se vuelve a ejecutar. De este modo, múltiples procesos periódicos pueden coexistir sobre un único temporizador sin interferir entre sí.

Este método permite, por ejemplo, activar un buzzer, actualizar el estado de un LED, escanear el teclado matricial y refrescar la pantalla LCD de forma concurrente y sincronizada, utilizando un solo temporizador interno. La principal ventaja de este enfoque radica en que elimina el uso de retardos activos (`_delay_ms()`), evitando el *polling* con-

tinuo y mejorando la eficiencia general del sistema. Además, al ser determinista y cooperativo, el programador conserva un control explícito sobre la prioridad y el momento de ejecución de cada tarea, lo cual resulta fundamental en sistemas con recursos de hardware limitados y requisitos de respuesta predecibles.

III. METODOLOGÍA

III-A. Materiales y Herramientas

- Microcontrolador ATmega328P (Arduino Uno)
- Sensor LDR y LED RGB
- Servomotor SG90
- Teclado matricial 4x4
- Pantalla LCD 16x2 con interfaz I2C
- Buzzers piezoeléctricos (activo y pasivo)
- Tira LED WS2812
- Resistencias, cables, protoboard
- Software: Microchip Studio, Proteus / PicSimLab, Python (para generación de secuencias del plotter)

III-B. Procedimiento general

Se dividió el sistema en cuatro módulos independientes (Plotter, Colores, Piano y Cerradura) y se abarcó cada consigna de manera individual.

III-B1. Plotter:

III-B2. Colores:

Idea general: — El circuito se implementó mediante un divisor resistivo conectado a una de las entradas analógicas del microcontrolador. El color identificado se replica visualmente en la tira de LED WS2812, y el servomotor se posiciona en el ángulo correspondiente (predefinido) al color detectado dentro de la hoja de referencia, integrando así un sistema de selección e identificación de color completamente automatizado.

Descomposición de colores: — El proceso de reconocimiento de colores se fundamenta en la descomposición de cada color en sus componentes primarias dentro del modelo RGB (*Red, Green, Blue*). Un color puede representarse como la combinación ponderada de estas tres intensidades, lo que permite su descripción en un espacio tridimensional.

Sin embargo, el sensor fotoresistivo (LDR, *Light Dependent Resistor*) utilizado en el sistema no distingue de forma individual las componentes cromáticas del espectro visible; únicamente mide la intensidad total de luz reflejada sobre su superficie. Si la iluminación se realiza con luz blanca, el sensor solo entrega una lectura proporcional a la cantidad total de luz reflejada, sin discriminar su composición espectral. Este fenómeno equivale a una percepción en escala de grises, dificultando la identificación precisa de colores.

Para resolver esta limitación, se empleó un diodo emisor de luz RGB como fuente de iluminación controlada. Al iluminar secuencialmente la superficie con luz roja, verde y azul, el sistema obtiene tres mediciones independientes

mediante el LDR. Dichos valores, convertidos por el módulo ADC (*Analog-to-Digital Converter*) del microcontrolador ATmega328P, representan las coordenadas (R, G, B) de un punto dentro de un espacio de color tridimensional (véase anexo VI-A2b).

Calibración y reconocimiento: — Dado que tanto la respuesta espectral del LDR como la emisión de los LED presentan variaciones no lineales, los valores obtenidos no son directamente proporcionales a las componentes RGB teóricas. No obstante, se implementa un proceso de calibración inicial para contrarrestar estos efectos, donde se miden manualmente los colores de referencia (rojo, verde, azul claro, violeta, morado, amarillo y blanco) y se almacenan sus valores de respuesta RGB de conversión analógica-digital.

Cada color de referencia calibrado se modela como un vector fijo en dicho espacio. Para identificar un color, se calcula la distancia euclidiana entre el vector de medición y cada uno de los vectores de referencia almacenados:

$$D = \sqrt{(R_m - R_i)^2 + (G_m - G_i)^2 + (B_m - B_i)^2} \quad (1)$$

donde (R_m, G_m, B_m) representan los valores medidos por el sensor y (R_i, G_i, B_i) corresponden a los valores calibrados de cada color de la hoja de referencia. El color identificado será aquel cuya distancia D sea mínima (véase anexo VI-A2f).

Servomotor y tira LED: — El servomotor se controla mediante el *Timer1* configurado en modo *Fast PWM* a 50 Hz, generando pulsos de entre 1 y 2 ms para posicionar el eje según el color identificado. Cada ángulo se calcula a partir del valor del registro *OCR1A*, que define el tiempo en alto del ciclo PWM (véase anexo VI-A2e).

La visualización del color se realiza con una tira de LED WS2812, donde cada diodo recibe datos digitales codificados en tres bytes (RGB). Para cumplir con los tiempos estrictos del protocolo, se emplearon instrucciones `asm volatile` que garanticen una sincronización precisa en la transmisión de pulsos. Así, al identificar un color, el sistema replica su tonalidad en la tira LED y mueve el servomotor al ángulo correspondiente, integrando una respuesta visual y mecánica simultánea (véase anexo VI-A2d).

USART: — A través de USART se comunican los valores del sensor y el color detectado hacia una interfaz serial. Se utiliza un buffer circular para transmisión continua sin bloqueo, y la función auxiliar *UTOA* convierte los datos numéricos del ADC en texto ASCII antes de su envío. De esta forma, el sistema comunica en tiempo real los valores (R, G, B) medidos, el color identificado y el ángulo del servomotor, facilitando el monitoreo y la calibración del sistema (véase anexo VI-A2g).

III-B3. Piano:

Notas musicales: — El sonido es una onda mecánica que se propaga a través de un medio elástico, producto de variaciones periódicas de presión. Estas ondas pueden clasificarse según su forma en senoidales, cuadradas, triangulares o diente de sierra, dependiendo del patrón temporal de oscilación. En los sistemas digitales, las señales más sencillas de generar son las ondas cuadradas, donde el voltaje alterna entre dos niveles definidos, representando los estados lógicos alto y bajo del microcontrolador.

Para producir sonido de manera electrónica, se emplean dispositivos piezoeléctricos denominados *buzzers*. Estos transductores convierten la energía eléctrica en vibraciones mecánicas audibles. Cuando el microcontrolador aplica una señal cuadrada a la entrada del buzzer, el material piezoeléctrico se deforma y contrae periódicamente, generando un sonido cuya frecuencia está directamente relacionada con la frecuencia de la señal de entrada.

En el microcontrolador ATmega328P, esta señal se genera mediante el uso de los temporizadores internos (*timers*), configurados para producir una onda cuadrada en un pin de salida. Al modificar la frecuencia de conmutación, es posible variar el tono percibido, ya que la frecuencia del sonido determina su altura musical. De esta manera, cada nota puede asociarse a una frecuencia específica según la escala. Por ejemplo, la nota *La4* corresponde a una frecuencia de 440 Hz, mientras que *Do4* equivale aproximadamente a 262 Hz (véase anexo VI-A3b).

Generación de frecuencias: — Para reproducir una nota musical, el microcontrolador debe generar una señal cuadrada con una frecuencia específica, determinada por el valor de comparación del temporizador. En el ATmega328P, dicha frecuencia depende de la relación entre la frecuencia del reloj del sistema (F_{CPU}), el valor de comparación *OCRx*, y el prescaler aplicado al temporizador. Este último actúa como un divisor de frecuencia, reduciendo la velocidad a la que el contador incrementa.

Elegir el prescaler adecuado resulta fundamental para asegurar que el valor de *OCRx* se mantenga dentro del rango permitido por el temporizador (por ejemplo, 8 bits en el *Timer0*). Si la frecuencia deseada es alta, se requiere un prescaler pequeño para mantener precisión; si es baja, un prescaler grande evita desbordamientos y permite generar tonos audibles.

En la práctica, el programa selecciona dinámicamente el prescaler que produce un valor de *OCR0A* válido para la nota solicitada (véase anexo VI-A3c). De esta forma, es posible generar un amplio rango de frecuencias musicales con un solo temporizador, manteniendo una relación estable entre tono y duración.

Reproducir música: — Una nota musical puede representarse computacionalmente mediante tres parámetros fundamentales: la frecuencia de oscilación, la duración temporal

durante la cual se mantiene activa y la duración inactiva o en silencio. Cuando el buzzer emite una secuencia ordenada de notas, cada una con su frecuencia, tiempo de activación, y silencio, se obtiene una melodía. En términos programáticos, una melodía se define como un conjunto de estructuras de datos que contienen pares (*frecuencia*, *tiempo_on*, *tiempo_off*), los cuales son procesados secuencialmente para generar la música deseada.

Para crear pistas de canciones se utilizó la herramienta *MIDI to Arduino Converter* [1], la cual permite convertir pistas MIDI al formato anteriormente mencionado, y almacenarlas dentro de la memoria del microcontrolador.

La reproducción simultánea de varios instrumentos en un entorno digital requiere el manejo de múltiples secuencias de notas en paralelo. Para lograrlo sin recurrir al uso de ciclos de espera activos (*polling*), se utilizan interrupciones asociadas a los temporizadores. De este modo, el microcontrolador puede controlar la duración y el inicio de cada nota de forma independiente y precisa, optimizando el uso del procesador y permitiendo la ejecución de tareas concurrentes, como la lectura de entradas o la comunicación serial, mientras la música continúa sonando de manera autónoma (véase anexo VI-A3d).⁹

Modos de funcionamiento y USART: — Para implementar el modo piano se decidió utilizar 8 pulsadores correspondientes a las 8 principales notas musicales. Cada uno de los pulsadores fue conectado a un pin del puerto D del microcontrolador, configurando los pines como entradas y pull-ups internas. Utilizando interrupciones externas por cambio de pin (*PCINT*) y lógica programática para debouncing, se implementó la funcionalidad de que mientras el pulsador está presionado, la nota sigue sonando.

Finalmente, a este sistema se integra funcionalidad USART para mostrar un menú al usuario por consola y permitir comandar el funcionamiento del sistema de manera sencilla. El usuario es presentado con 3 opciones: [1]: Reproducir canción 1, [2]: Reproducir canción 2, [P]: Cambiar a modo piano. Mediante variables de control se deshabilita la funcionalidad de piano durante la reproducción de canciones, y vice versa, al cambiar al modo piano se detiene y reinician las pistas que se estaban reproduciendo (véase anexo VI-A3e).

En síntesis, el sistema desarrollado utiliza un buzzer piezoeléctrico controlado por los temporizadores del ATmega328P para reproducir notas musicales definidas por su frecuencia y duración. A través de la programación de secuencias almacenadas en memoria, es posible interpretar melodías completas y gestionar la reproducción de distintas canciones sin intervención del usuario durante la ejecución. Utilizando USART se puede comandar al programa para reproducir diferentes pistas guardadas o cambiar a modo piano donde 8 pulsadores reproducen las 8 notas principales.

III-B4. Cerradura:

Idea general: — El sistema de cerradura electrónica desarrollado tiene como objetivo controlar el acceso mediante una contraseña numérica almacenada en memoria no volátil. Inicialmente, el dispositivo se encuentra en estado de *candado cerrado*. Cuando el usuario ingresa la contraseña correcta, el sistema cambia al estado de *candado abierto*, permitiendo el acceso. En caso de introducir una contraseña incorrecta tres veces consecutivas, se activa una alarma acústica a través de un buzzer, indicando un intento de acceso no autorizado.

El sistema también permite modificar la contraseña almacenada. Para ello, el usuario debe ingresar primero la contraseña actual y, tras su validación, definir una nueva contraseña de entre cuatro y seis dígitos. Esta nueva clave se almacena permanentemente en la memoria EEPROM del microcontrolador, garantizando su persistencia incluso después de un apagado o reinicio del sistema.

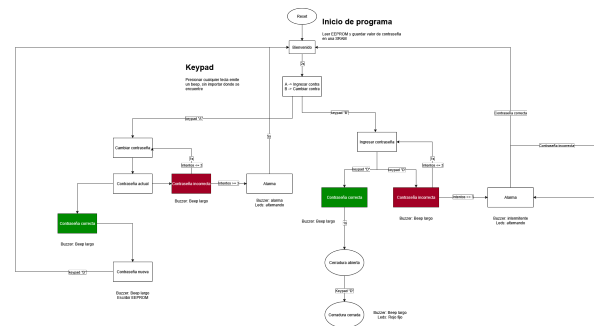


Figura 1. Diagrama de flujo de programa de cerradura. Fuente: elaboración propia

Interfaz de usuario: — La cerradura dispone de una interfaz de usuario compuesta por una pantalla LCD de 16x2, un teclado matricial 4x4 y tres indicadores luminosos (LED verde, LED rojo y alarma sonora). En este contexto, la interfaz constituye el medio de comunicación entre el usuario y el sistema, mostrando mensajes informativos y recibiendo acciones por medio del teclado. Cada interacción del usuario genera una respuesta visual o acústica distinta, representando así un flujo de diálogo entre ambos.

El sistema se diseñó siguiendo una lógica de *máquina de estados finitos*, en la que cada modo de operación (menú principal, ingreso de contraseña, cambio de clave, acceso autorizado, alarma, etc.) representa un estado. Las transiciones entre estados se producen en función de las acciones del usuario y las condiciones del sistema. Este enfoque facilita la gestión de comportamientos complejos, simplifica el control del flujo de ejecución y mejora la legibilidad del código (véase anexo VI-A4b).

Teclado matricial: — El ingreso de datos se realiza mediante un teclado matricial 4x4 que combina filas y columnas, optimizando el uso de pines del microcontrolador.

Las filas se configuran como salidas y las columnas como entradas con resistencias de *pull-up* activadas. El proceso de lectura consiste en activar una fila a nivel bajo (0) mientras las demás permanecen en alto (1); si alguna tecla de esa fila se encuentra presionada, la columna correspondiente cambia a nivel bajo, permitiendo identificar la intersección entre fila y columna (véase anexo VI-A4c).

Cada tecla se asocia a un carácter según su posición dentro de la matriz, lo que permite retornar el valor numérico o simbólico correspondiente. Para evitar falsas detecciones debido al rebote mecánico de los contactos, se aplica una rutina de *debouncing* por software basada en pequeños retardos temporizados.

Planificador de tareas: — Durante el funcionamiento, el sistema debe ejecutar múltiples tareas de forma paralela: reproducción de sonidos, manejo de alarmas, parpadeo de LEDs, lectura del teclado y actualización de la pantalla LCD. Sin embargo, el ATmega328P dispone de un número limitado de temporizadores, por lo que no es posible asignar un temporizador independiente a cada tarea.

Para resolver esta limitación, se implementó un esquema de ejecución basado en un *planificador de tareas* o *task scheduler* de propósito general. Se utiliza un solo temporizador configurado para generar interrupciones periódicas, incrementando un contador global de 32 bits que actúa como referencia temporal (en milisegundos). Cada tarea compara el tiempo actual con el instante programado de su próxima ejecución, y si se cumple el intervalo, se ejecuta la acción correspondiente. De este modo, múltiples eventos temporizados pueden coexistir sin emplear retardos bloqueantes ni técnicas de *polling* (véase anexo VI-A4d).

Almacenamiento en EEPROM: — Para asegurar que la contraseña permanezca almacenada tras un apagado, se utiliza la memoria EEPROM interna del ATmega328P. Esta memoria no volátil permite conservar los datos durante décadas sin alimentación eléctrica, aunque posee un número limitado de ciclos de escritura (aproximadamente 10^5 operaciones por celda). Por ello, el sistema únicamente escribe en EEPROM cuando el usuario confirma un cambio de contraseña, minimizando el desgaste de la memoria.

La lectura y escritura se realizan mediante las funciones de la biblioteca estándar `avr/eeprom.h`, que permiten transferir cadenas de caracteres directamente desde y hacia direcciones específicas de la EEPROM. Así, el sistema recupera la contraseña almacenada al inicio del programa y la compara con la ingresada por el usuario durante la operación normal (véase anexo VI-A4e).

En conjunto, la cerradura electrónica combina la interacción mediante teclado y pantalla LCD, la gestión de tareas en tiempo real y el almacenamiento persistente de datos en memoria no volátil EEPROM. El uso de una máquina de estados y un planificador temporal basado en un único

temporizador permite controlar de manera eficiente múltiples procesos simultáneos sin bloquear la ejecución principal del programa.

IV. RESULTADOS

IV-A. Resultados por módulo

IV-A1. **Plotter:** —

IV-A2. **Colores:** —

Cuadro I
VALORES DE REFERENCIA RGB PARA CADA COLOR CALIBRADO.

Color	R	G	B
Morado	216	157	274
Rojo	206	149	262
Amarillo	196	99	105
Verde	272	192	152
Azul Claro	151	153	122
Violeta	253	275	338
Blanco	110	93	91

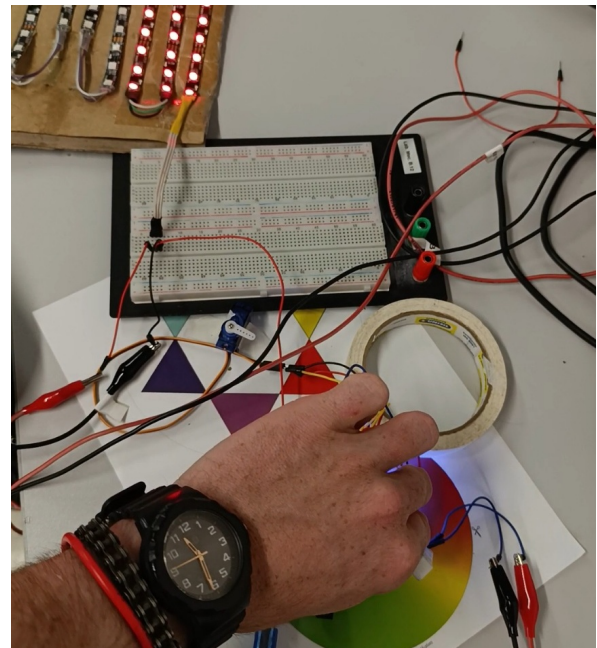


Figura 2. Identificación de color efectiva. Fuente: elaboración propia

Utilizando una fuente externa el sistema es considerablemente estable. Como el servomotor posee un pico de consumo en momentos de arranque, si este es alimentado solamente con el voltaje del Arduino, algunos colores se volvían inestables y creaba fallas en la medición, haciendo un ciclo de retroalimentación.

Otra consideración fue que el sistema también es dependiente en cierta medida a la cantidad de luz ambiental del lugar en el que se encuentre. La misma tira de LEDs al iluminarse a veces causaba falsas medidas, se solventó esto bloqueando la luz de la tira hacia el sensor.

También, al implementar la impresión de datos a través de USART, se observó una ralentización considerable del

funcionamiento del programa debido al tiempo que le tomaba enviarse a los datos para cada ciclo. Al quitar esta funcionalidad el sistema lograba un aumento significativo en su velocidad de operación.

IV-A3. *Piano*: —

IV-A4. *Cerradura*: —

IV-B. *Evidencias gráficas y mediciones*

IV-C. *Comentario general*

V. CONCLUSIONES

V-A. *Plotter*

V-B. *Colores*

- La comunicación por USART fue una limitación para la velocidad del sistema. Podría mejorarse reduciendo la cantidad de datos enviados o usando interrupciones para hacerlo más fluido.
- Sería útil agregar un modo de calibración donde el usuario pueda registrar los valores de referencia de cada color, adaptando el sistema a distintas condiciones de luz.
- Se podría usar un servomotor de 360° para aprovechar todos los colores disponibles en la rueda.

V-C. *Piano*

- Sería interesante agregar más canciones al repertorio.
- Se podría mejorar la coordinación entre las pistas, ya que con el tiempo tienden a desfasarse un poco.
- También sería bueno incluir más notas en el teclado para ampliar el rango musical.

V-D. *Cerradura*

- Agregar un botón para ver la contraseña escrita antes de confirmar ayudaría al usuario sin afectar la seguridad.
- Implementar un solenoide real permitiría que el sistema abra y cierre físicamente el cerrojo.
- En la simulación de PicSimLab el reinicio no siempre funciona correctamente. A veces es necesario usar el botón “DEBUG” o reiniciar el programa. Sin embargo, en el dispositivo real el sistema funciona correctamente.

REFERENCIAS

- [1] ShivamJoker, “Midi to arduino converter,” <https://arduinomidi.netlify.app/>, 2024, herramienta en línea para convertir archivos MIDI a código Arduino.
- [2] Microchip Technology Inc. Atmega328p datasheet. [Online]. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf
- [3] circuito.io. (2018) Arduino uno pinout diagram. [Online]. Available: <https://www.circuito.io/blog/arduino-uno-pinout/>
- [4] ARXterra. Addressing modes — 8-bit avr instruction set. [Online]. Available: <https://www.arxterra.com/3-addressing-modes/>
- [5] Software Particles. (2024, Apr.) Learn how an 8x8 led display works and how to control it using an arduino. Figura: 8x8 LED matrix pin mapping (imagen del artículo). [Online]. Available: <https://softwareparticles.com/learn-how-an-8x8-led-display-works-and-how-to-control-it-using-an-arduino/>
- [6] Carpeta del laboratorio (google drive). Carpeta compartida del laboratorio. [Online]. Available: <https://drive.google.com/drive/u/0/folders/1fP0aIlLozXeapRgDPDNWT1TRhAYr1PPPT>

- [7] Microchip Community (AVR Freaks). Avr freaks — comunidad de desarrolladores avr. Foros técnicos y soluciones prácticas sobre AVR. [Online]. Available: <https://www.avrfreaks.net/>
- [8] J. Ganssle. A guide to debouncing. Referencia clásica para anti-rebote en pulsadores/encoders. [Online]. Available: <https://www.ganssle.com/debouncing.htm>
- [9] G. Schmidt. (2021, Sep.) Beginners introduction to the assembly language of atmel-avr-microprocessors. Tutorial de avr-asm-tutorial.net (revisión septiembre 2021). [Online]. Available: <https://kitsandparts.com/tutorials/assemblers/BeginnersAVRasm.pdf>
- [10] T. Redelberger. (2019, Apr.) Avr assembler for complex projects. Versión 0.4 (2019-04-06). [Online]. Available: https://web222.webclient5.de/doc/swdev/avrasm/advavrasm2/AdvancedUseAVRASM2_en_20190406.pdf
- [11] Laboratorio de Microcontroladores, UTEC, “Repositorio de laboratorio de microcontroladores (tec.micro),” <https://github.com/MateoLecuna/Tec.Micro>, 2025, accedido el 26 de septiembre de 2025.

VI. ANEXOS

VI-A. *Fragmentos de código*

VI-A1. *Plotter*: —

VI-A2. *Colores*: —

VI-A2a. *Librerías utilizadas*: —

```
#define F_CPU 16000000UL

#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
#include <string.h>
```

Listing 1. Librerías utilizadas

VI-A2b. *Lectura de colores*: —

```
uint8_t led_state = 0;
uint16_t adc_sample[] = {0,0,0};

// Leer adc
uint16_t adc_read(uint8_t channel) {
    ADMUX = (ADMUX & 0xF0) | (channel & 0x0F);
    ADCSRA |= (1 << ADSC);
    while (ADCSRA & (1 << ADSC));
    return ADC;
}

// Establecer color del led rgb (iluminador)
void rgb_set(uint8_t r, uint8_t g, uint8_t b) {
    PORTB = (PORTB & ~(1 << RED) | (1 << GREEN) | (1 << BLUE)) |
        ((r << RED) | (g << GREEN) | (b << BLUE));
}

// Ciclo de medicion RGB
void rgb_read(void) {

    switch(led_state) {
        case 0:
            rgb_set(1,0,0);
            adc_sample[0] = adc_read(0); // Rojo
            break;

        case 1:
            rgb_set(0,1,0);
            adc_sample[1] = adc_read(0); // Verde
            break;

        case 2:
            rgb_set(0,0,1);
```



```

        adc_sample[2] = adc_read(0); // Azul
        break;
    }
}

```

Listing 2. Lectura de colores

VI-A2c. Convertidor a string: —

```

// Convierte un valor entero sin signo en un
// string
void UTOA(uint16_t value, char *buffer) {
    char temp[6];
    int i = 0, j = 0;

    if (value == 0) {
        buffer[0] = '0';
        buffer[1] = '\0';
        return;
    }

    // Convert digits to temp buffer (reversed)
    while (value > 0 && i < sizeof(temp) - 1) {
        temp[i++] = (value % 10) + '0';
        value /= 10;
    }

    // Reverse digits into final buffer
    while (i > 0) buffer[j++] = temp[--i];
    buffer[j] = '\0';
}

```

Listing 3. Convertidor de unsigned integer a string

VI-A2d. Manejo de tira led WS2812: —

```

void send_bit(uint8_t bitVal) {
    if (bitVal) {
        PORTD |= (1<<LED_PIN);
        asm volatile (
            "nop\n\t""nop\n\t""nop\n\t""nop\n\t""nop\n\t"
            "\t"
            "nop\n\t""nop\n\t""nop\n\t""nop\n\t";

        PORTD &= ~(1<<LED_PIN);

        asm volatile (
            "nop\n\t""nop\n\t""nop\n\t""nop\n\t";

        } else {
        PORTD |= (1<<LED_PIN);
        asm volatile (
            "nop\n\t""nop\n\t""nop\n\t";

        PORTD &= ~(1<<LED_PIN);
        asm volatile (
            "nop\n\t""nop\n\t""nop\n\t""nop\n\t""nop\n\t"
            "\t"
            "nop\n\t""nop\n\t""nop\n\t""nop\n\t""nop\n\t"
            "\t");
        }
    }

    // Envia un Byte a la tira de leds
    // Utiliza cli y sei para un timing preciso
    void send_byte(uint8_t byte) {
        cli();
        for (uint8_t i = 0; i < 8; i++) {
            send_bit(byte & 0x80); // Enviar msb
            primero
            byte <<= 1;
        }
    }
}

```

```

    sei();
}

void ws2812_send_pixel(uint8_t r, uint8_t g,
    uint8_t b) {
    // Cambiar orden dependiendo de formato de
    // tira led o matriz
    send_byte(g);
    send_byte(b);
    send_byte(r);
}

// Encender n leds del mismo color
void ws2812_fill(uint8_t r, uint8_t g, uint8_t b,
    uint16_t n) {
    cli();
    for (uint16_t i = 0; i < n; i++) {
        ws2812_send_pixel(r, g, b);
    }
    sei();
    ws2812_show();
}

// Establecer colores predeterminados
void led_strip_set_color(uint8_t color_id) {
    uint8_t r = 0, g = 0, b = 0;

    switch (color_id) {
        case 1: // Rojo
            r = 255; g = 0; b = 0;
            break;

        case 2: // Amarillo
            r = 255; g = 100; b = 0;
            break;

        case 3: // Verde
            r = 0; g = 255; b = 0;
            break;

        case 4: // Azul claro
            r = 0; g = 255; b = 255;
            break;

        case 5: // Violeta
            r = 100; g = 0; b = 100;

            break;

        case 6: // Morado
            r = 200; g = 0; b = 75;

            break;

        case 7: // Blanco
            r = 255; g = 255; b = 255;
            break;

        default: // Apagar LED
            r = g = b = 0;
            break;
    }

    ws2812_fill(r, g, b, 50);
}

```

Listing 4. Manejo de tira led WS2812

VI-A2e. Movimiento de servomotor: —

```

// Establecer angulo en el servomotor
void servo_set_angle(uint8_t angle) {
    uint16_t pulse = 1000 + ((uint32_t)angle *

```



```

    4000) / 180;
    OCR1A = pulse;
}

```

Listing 5. Movimiento de servomotor

VI-A2f. Reconocimiento de colores: —

```

// Mapeado de colores RGB
typedef struct {
    const char *name;
    uint16_t r, g, b;
} ColorRef;

const ColorRef color_refs[] = {
    {"MORADO",    216, 157, 274},
    {"ROJO",      206, 149, 262},
    {"AMARILLO",  196, 99, 105},
    {"VERDE",     272, 192, 152},
    {"AZUL CLARO", 151, 153, 122},
    {"VIOLETA",   253, 275, 338},
    {"BLANCO",    110, 93, 91},
};

// Identificar color a partir de valores rgb.
const char* identify_color(
    uint16_t r,
    uint16_t g,
    uint16_t b
) {
    uint32_t best_dist = 0xFFFFFFFF;
    const char *best_name = "UNKNOWN";

    for (uint8_t i = 0; i < NUM_COLORS; i++) {
        int32_t dr = (int32_t)r - color_refs[i].r;
        int32_t dg = (int32_t)g - color_refs[i].g;
        int32_t db = (int32_t)b - color_refs[i].b;
        uint32_t dist = dr*dr + dg*dg + db*db;

        if (dist < best_dist) {
            best_dist = dist;
            best_name = color_refs[i].name;
        }
    }
    return best_name;
}

```

Listing 6. Reconocimiento de colores

VI-A2g. Comunicación por USART: —

```

// Imprimir datos parseados por usart
void print_data(const char *color_name) {
    char buf[10];

    // Find by name
    uint16_t ref_r = 0, ref_g = 0, ref_b = 0;
    for (uint8_t i = 0; i < NUM_COLORS; i++) {
        if (strcmp(color_name, color_refs[i].name)
            == 0) {
            ref_r = color_refs[i].r;
            ref_g = color_refs[i].g;
            ref_b = color_refs[i].b;
            break;
        }
    }

    // Delta
    int16_t dR = (int16_t)adc_sample[0] - ref_r;
    int16_t dG = (int16_t)adc_sample[1] - ref_g;

```

```

    int16_t dB = (int16_t)adc_sample[2] - ref_b;

    // Printing
    usart_write_str("Fotocelda: ");
    UTOA(adc_sample[0], buf); usart_write_str(buf)
    ;
    usart_write_str(" ");
    UTOA(adc_sample[1], buf); usart_write_str(buf)
    ;
    usart_write_str(" ");
    UTOA(adc_sample[2], buf); usart_write_str(buf)
    ;

    usart_write_str(" Color detectado: ");
    usart_write_str(color_name);

    usart_write_str(" Valor establecido: [");
    UTOA(ref_r, buf); usart_write_str(buf);
    usart_write_str(",");
    UTOA(ref_g, buf); usart_write_str(buf);
    usart_write_str(",");
    UTOA(ref_b, buf); usart_write_str(buf);
    usart_write_str("]");

    usart_write_str(" Delta: [");
    if (dR >= 0) usart_write_str("+");
    else usart_write_str("-");
    UTOA((dR >= 0) ? dR : -dR, buf);
    usart_write_str(buf);
    usart_write_str(",");

    if (dG >= 0) usart_write_str("+");
    else usart_write_str("-");
    UTOA((dG >= 0) ? dG : -dG, buf);
    usart_write_str(buf);
    usart_write_str(",");

    if (dB >= 0) usart_write_str("+");
    else usart_write_str("-");
    UTOA((dB >= 0) ? dB : -dB, buf);
    usart_write_str(buf);
    usart_write_str("]\r\n");
}

```

Listing 7. Comunicación por USART

VI-A2h. Bucle principal: —

```

// programa principal
int main(void) {
    ws2812_init();
    usart_init();
    adc_init();
    rgb_init();
    servo_init();
    sei();

    while (1) {
        rgb_read();
        _delay_ms(50);
    }
}

```

Listing 8. Bucle principal

VI-A3. Piano: —

VI-A3a. Librerías utilizadas: —

```

#define F_CPU 16000000UL
#include <avr/io.h>

```

```
#include <avr/interrupt.h>
#include <avr/pgmspace.h>
```

Listing 9. Librerías utilizadas

VI-A3b. Guardado de notas musicales: —

```
#define G4 392
#define Fb4 370
#define E4 330
#define D4 294
// ...
```

Listing 10. Selección dinámica de prescaler

VI-A3c. Selección dinámica de prescaler: —

```
void playFrequencyA(uint16_t freq) {
    if (!freq) return;

    DDRD |= (1 << PORTD6);

    // Elegir prescaler adecuado para esa
    frecuencia

    uint8_t presc_bits = 0;
    uint16_t ocr = 0;

    const uint16_t presc_list[] = {8, 64, 256,
    1024};
    const uint8_t bits_list[] = {0b010, 0b011, 0
    b100, 0b101};

    for (uint8_t i = 0; i < 4; i++) {
        ocr = (F_CPU / (2UL * presc_list[i] * freq
        )) - 1;
        if (ocr <= 255) {
            presc_bits = bits_list[i];
            break;
        }
    }

    TCCR0A = (1 << COM0A0) | (1 << WGM01);
    TCCR0B = presc_bits;
    OCR0A = (uint8_t)ocr;
}

// Dejar de reproducir frecuencia buzzer 1
void stopFrequencyA(void) {
    TCCR0A = 0;
    TCCR0B = 0;
    DDRD |= (1 << PORTD6);
    PORTD &= ~(1 << PORTD6);
}
```

Listing 11. Selección dinámica de prescaler

VI-A3d. Reproducción de pistas: —

```
const int midiA[349][3] PROGMEM = {
    {G4, 252, 0},
    {Fb4, 252, 0},
    {E4, 252, 0},
    {E4, 252, 0},
    // ...
}

void read_midi_event(const int (*track)[3],
    uint16_t index, int out_values[3]) {
```

```
    for (uint8_t i = 0; i < 3; i++) {
        out_values[i] = pgm_read_word(&(track[
        index][i]));
    }
}

// Reproducir melodía o track en buzzer 1
void playTrackA(void) {
    int values[3];
    if (song == 0){
        read_midi_event(midiC, indexA++, values);
    } else {
        read_midi_event(midiA, indexA++, values);
    }

    uint16_t freq = values[0];
    maxCountAon = values[1];
    maxCountAoff = values[2];

    playFrequencyA(freq);
    enableCountAon = 1;
}

ISR(TIMER1_OVF_vect){
    TCNT1 = 65536 - 250; // 1ms preload

    // Debouncing
    if (debounce_active) { // 1 ms debounce
        PCICR |= (1 << PCIE1) | (1 << PCIE2);
        debounce_active = 0;
    }

    // Note playing
    if (enableCountAon) {
        if (++countA > maxCountAon) {
            eventAon = 1;
        }

        } else if (enableCountAoff){
        if (++countA > maxCountAoff){
            eventAoff = 1;
        }
    }

    if (enableCountBon) {
        if (++countB > maxCountBon) {
            eventBon = 1;
        }

        } else if (enableCountBoff){
        if (++countB > maxCountBoff){
            eventBoff = 1;
        }
    }
}
```

Listing 12. Reproducción de pistas

VI-A3e. Manejo de estados por USART: —

```
// Manejo de cambio de estados de usart
void handleUSART(uint8_t character){
    if (character == '1'){
        mode = 1;
        eventAoff = 1;
        eventBoff = 0;

        song = 0;

        eventAon = 0; // Encender track A
        indexA = 0; // Posición track A
        countA = 0; // Conteo de overflow de notas
        de track A
    }
```

```

    maxCountAon = 0; // Maximo conteo de
    overflow encendido en A
    maxCountAoff = 0; // Maximo conteo de
    overflow apagado en A
    enableCountAon = 0; // Habilidad conteo de
    encendido en A
    enableCountAoff = 0; // Habilidad conteo
    de apagado en A

    eventBon = 0; // Encender track B
    indexB = 0; // Posicion track B
    countB = 0; // Conteo de overflow de notas
    de track B
    maxCountBon = 0; // Maximo conteo de
    overflow encendido en B
    maxCountBoff = 0; // Maximo conteo de
    overflow apagado en B
    enableCountBon = 0; // Habilidad conteo de
    encendido en B
    enableCountBoff = 0; // Habilidad conteo
    de apagado en B

    PCICR &= ~(1 << PCIE1) | (1 << PCIE2));
    stopFrequencyB();

} else if (character == '2'){
    mode = 1;
    eventAoff = 1;
    eventBoff = 1;

    song = 1;

    eventAon = 0; // Encender track A
    indexA = 0; // Posicion track A
    countA = 0; // Conteo de overflow de notas
    de track A
    maxCountAon = 0; // Maximo conteo de
    overflow encendido en A
    maxCountAoff = 0; // Maximo conteo de
    overflow apagado en A
    enableCountAon = 0; // Habilidad conteo de
    encendido en A
    enableCountAoff = 0; // Habilidad conteo
    de apagado en A

    eventBon = 0; // Encender track B
    indexB = 0; // Posicion track B
    countB = 0; // Conteo de overflow de notas
    de track B
    maxCountBon = 0; // Maximo conteo de
    overflow encendido en B
    maxCountBoff = 0; // Maximo conteo de
    overflow apagado en B
    enableCountBon = 0; // Habilidad conteo de
    encendido en B
    enableCountBoff = 0; // Habilidad conteo
    de apagado en B

    PCICR &= ~(1 << PCIE1) | (1 << PCIE2));
    stopFrequencyA();

} else if (character == 'P'){
    mode = 0;
    eventAoff = 0;
    eventBoff = 0;

    eventAon = 0; // Encender track A
    indexA = 0; // Posicion track A
    countA = 0; // Conteo de overflow de notas
    de track A
    maxCountAon = 0; // Maximo conteo de
    overflow encendido en A
    maxCountAoff = 0; // Maximo conteo de
    overflow apagado en A
    enableCountAon = 0; // Habilidad conteo de

```

```

    encendido en A
    enableCountAoff = 0; // Habilidad conteo
    de apagado en A

    eventBon = 0; // Encender track B
    indexB = 0; // Posicion track B
    countB = 0; // Conteo de overflow de notas
    de track B
    maxCountBon = 0; // Maximo conteo de
    overflow encendido en B
    maxCountBoff = 0; // Maximo conteo de
    overflow apagado en B
    enableCountBon = 0; // Habilidad conteo de
    encendido en B
    enableCountBoff = 0; // Habilidad conteo
    de apagado en B

    startDebounceTimer();
    stopFrequencyA();
    stopFrequencyB();
}
}

```

Listing 13. Manejo de estados por USART

VI-A3f. Programa principal: —

```

// Programa principal
int main(void) {
    timer1_init();
    usart_init_9600();
    init_piano_buttons();
    sei();

    usart_write_str("Elija una opcion:\r\n");
    usart_write_str("[1] Dragon Ball - Cha-La Head
    -Cha-La\r\n");
    usart_write_str("[2] Portal - Still alive\r\n"
    );
    usart_write_str("[P] Modo piano\r\n");

    while (1){
        if (mode == 0){
            piano_mode();
        } else if (mode == 1){
            song_mode();
        }
        uint8_t c;
        if (usart_read_try(&c)) {
            handleUSART(c);
        }
    }
}

```

Listing 14. Programa principal

VI-A4. Cerradura: —

VI-A4a. Librerias para lcd: —

```

#include "i2c_master.h"
#include "i2c_master.c"
#include "liquid_crystal_i2c.h"
#include "liquid_crystal_i2c.c"

int main(void){
    LiquidCrystalDevice_t device = lq_init(0x27,
    16, 2, LCD_5x8DOTS);
    lq_turnOnBacklight(&device);
    char welcomeText[] = "Bienvenido!";
    lq_print(&device, welcomeText);
    while (1);
}

```

```
}
```

Listing 15. Librerías utilizadas para pantalla LCD

VI-A4b. Lógica de estados: —

```
typedef enum { UI_MENU, UI_INGRESO,
  UI_CAMBIO_ACTUAL, UI_CAMBIO_NUEVA, UI_ABIERTO,
  UI_ALARMA } ui_state_t;

int main(void) {
  while(1) {
    char key = keypad_scan();
    if (key) {
      switch (ui_state)
      {
        case UI_MENU: break;
        case UI_INGRESO: break;
        case UI_CAMBIO_ACTUAL: break;
        case UI_CAMBIO_NUEVA: break;
        case UI_CAMBIO_ALARMA: break;
        case UI_CAMBIO_ABIERTO: break;
      }
    }
  }
}
```

Listing 16. Lógica de estados

VI-A4c. Lectura multiplexada de keypad: —

```
// Mapeo de keypad
const char keypad[4][4] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};

char keypad_scan(void) {
  if (!keypad_enable) return 0;

  uint8_t row, col;
  uint8_t cols;
  static uint8_t prevKey; // store for later

  for (row = 0; row < 4; row++) {
    PORTD = (PORTD | 0xF0) & ~(1 << (row + 4));
    _delay_us(5);
    cols = PIND & 0x0F;

    for (col = 0; col < 4; col++) {
      if (!(cols & (1 << col)) ) {
        if ((prevKey == keypad[row][col]))
          return 0;

        keypad_debounce_ms(200);
        prevKey = keypad[row][col];
        return keypad[row][col];
      }
    }
  }
  prevKey = 0;
  return 0;
}
```

Listing 17. Función de lectura multiplexada de keypad

VI-A4d. Implementación de tareas: —

```
uint32_t millis_counter = 0;
```

```
uint32_t keypad_on_at = 0;
uint8_t keypad_enable = 1;

ISR(TIMER0_OVF_vect) {
  millis_counter++;
}

uint32_t millis_now(void) {
  uint32_t m;
  cli(); // disable interrupts
  m = millis_counter;
  sei(); // re-enable
  return m;
}

void keypad_debounce_ms(uint16_t delay_ms) {
  keypad_enable = 0;
  keypad_on_at = millis_now() + delay_ms;
}

void keypad_task(void) {
  if (!keypad_enable && (millis_now() >
    keypad_on_at)) {
    keypad_enable = 1;
  }
}

int main(void) {
  // main code ...
  while (1) {
    keypad_task();
    // loop code ...
  }
}
```

Listing 18. Implementación de tareas

VI-A4e. Escritura y lectura de EEPROM: —

```
#include <avr/eeprom.h>

#define MAX_PASSWORD_LENGTH 6
#define EEPROM_MAGIC 0x42

uint8_t EEMEM ee_magic;
char EEMEM ee_password[MAX_PASSWORD_LENGTH + 1];

char storedPassword[MAX_PASSWORD_LENGTH + 1] = "
  123456";
char typedPassword[MAX_PASSWORD_LENGTH + 1];

void eeprom_load_password(void) {
  if (eeprom_read_byte(&ee_magic) !=
    EEPROM_MAGIC) {

    eeprom_update_block(
      storedPassword,
      ee_password,
      sizeof(storedPassword)
    );

    eeprom_update_byte(&ee_magic, EEPROM_MAGIC
  );
  } else {
    eeprom_read_block(
      storedPassword,
      ee_password,
      sizeof(storedPassword)
    );
  }
}
```

```

void eeprom_save_password(const char *pwd) {
    char buf[MAX_PASSWORD_LENGTH + 1];
    strncpy(buf, pwd, MAX_PASSWORD_LENGTH);
    buf[MAX_PASSWORD_LENGTH] = '\0';
    eeprom_update_block(buf, ee_password, sizeof(
    buf));
}

```

Listing 19. Escritura y lectura de EEPROM

VI-A4f. Tarea de alarma: —

```

void alarm_task(void) {
    if (!alarm_active) return;
    uint32_t now = millis_now();

    if (now > alarm_until){
        alarm_active = 0;
        PORTB &= ~(1<<PORTB5);
        led_red_on();
        return;
    }

    if (now > alarm_next_toggle){
        alarm_next_toggle = now + ALARM_TOGGLE_MS;
        if (alarm_phase){
            led_red_on();
        } else {
            led_green_on();
        }
        alarm_phase ^= 1;
        buzzer_beep(100);
    }
}

```

Listing 20. Tarea de alarma